Closure

1. a closure is a function that allows you to have access to variables even after the scope is closed. It also keep tracks of the data
2. A closure is a combination of a function and lexical environment

"JavaScript variables can belong to the local or global scope. Global variables can be made local (private) with closures... JavaScript supports nested functions. Nested functions have access to the scope 'above' them." -- https://www.w3schools.com/js/js_function_closures.asp

Closure with Nested Functions where the inner function gets a value from the outer function

Declare a function `outerFunc`, with a parameter `num1`.

```
function outerFunc(num1) {
}
```

Inside `outerFunc`, declare another function, `innerFunc`. This nested function has `num2` as its parameter:

```
function outerFunc(num1) {

        function innerFunc(num2) {

        }
}
```

We want `innerFunc` to return the sum of the numbers:

```
function outerFunc(num1) {

        function innerFunc(num2) {
            return num1 + num2;
        }
}
```

We want `outerFunc` to return `innerFunc`. The value of `num1` is available only to this returned function. The pre-set value of `num1` is saved in a special scope, or lexical envionment, called a "closure".

```
function outerFunc(num1) {

        function innerFunc(num2) {
            return num1 + num2;
        }
        // pre-set num1 value is "closure"
```

```
        return innerFunc;
}
```

Call `outerFunc`, passing it a value of `5`. The `5` sets the value of `num1` which is used to do the math in the `innerFunc`.

the closure is a memory "packet"

The returned value `innerFunc` is returned and saved to a variable, `inFunc`, which is a copy of `innerFunc`, except with the value of `num1` pre-set to `5`. Only `inFunc` has access to this `5`, which is saved to a special area of memory, called a "packet". This is the nature of a "closure" : access to a variable after a scope is closed.

```
let inFunc = outerFunc(5);
```

Now, when `inFunc` is called, it just needs `num2` passed in, so that it can add that to the `5` from the closure. The function call returns the sum, so save that to a variable called `sum`:

```
let sum = inFunc(8);
```

Finally, output the sum:

```
console.log(sum); // 13
```

Let's try a closure with strings. It has the same nested function structure as the previous example. It's just that it concatenates a string, rather than adds mumbers.

```
function exteriorFunc(str1) {
       function interiorFunc(str2) {
           return str1 + ' ' + str2;
       }
       return interiorFunc;
}
```

Call the outer function, providing the first part of the string to be concatenated:

```
let intFunc = exteriorFunc('Hello');
```

The function call returns the inner function, with `str1` pre-set to 'Hello'. The variable we save the function, `intFunc`, is a copy of `interiorFunc`. Output `intFunc` so we can see that it is an exact clone of

`interiorFunc`:

```
    console.log(intFunc);
```

closure concatenating strings

So, where is the 'Hello', anyway? It is the special, memory "packet" that is neither the global scope nor any local scope. Only the `intFunc` has access to the 'Hello'. This "closed scope access" *is* the "closure".

```
    let str = intFunc('World');
    console.log(str); // Hello World
```

closure with arrays

Let's try a closure with arrays. The also uses the nested function format. The difference is the inner function concatenate the array passed to the outer function with the function toward the inner function, and then is sorts them alphabetically.

```
    const citrusFruits = ['lemon', 'tangerine', 'orange', 'grapefruit',
'lime'];

    const tropicalFruits = ['pineapple', 'banana', 'mango', 'papaya',
'kiwi'];

    function outsideFunc(arr1) {

            function insideFunc(arr2) {
               // concatenate the two arrays and then sort them
               return arr1.concat(arr2).sort();
            }

            return insideFunc;

    }

    let insiderFunc = outsideFunc(citrusFruits);
    let comboArr = insiderFunc(tropicalFruits);
    console.log(comboArr);
```

Closure exercises:

1. Make a function/closure (nested functions), where each function has 1 parameter, both numbers.

- The outer function's params is squared (multiplied by itself).
- The outer function returns the inner function.

- The inner function also squares its param.
- The inner function returns the difference of the squares, that is, the outer function square minus the inner function square.
- Return the absloute value. So, if the difference is negative, make it positive. To get the absolute value of a number, pass it to the Math.abs() method.

2. Create a function/closure that returns two functions - in this function it has a variable called totalScore `let totalScore = 0` - scores - each time this function is called it adds to the totalScore and return `Kobe scores 5 points` - totalScore - this function returns the name and the total scores `Kobe's total score: 50`

Closure exercises SOLUTION:

1. Make a function/closure (nested functions), where each function has 1 parameter, both numbers.

- The outer function's params is squared (multiplied by itself).
- The outer function returns the inner function.
- The inner function also squares its param.
- The inner function returns the difference of the squares, that is, the outer function square minus the inner function square.
- Return the absloute value. So, if the difference is negative, make it positive. To get the absolute value of a number, pass it to the Math.abs() method.

```
// Math.abs() returns absolute value
// Math.abs(-7) = 7;
function outerFunction(n1) {

function innerFunction(n2) {
      return Math.abs((n1 * n1) - (n2 * n2));
}

return innerFunction;

}

let outFunction = outerFunction(12);
let diffSq = outFunction(5);
console.log(diffSq); // 144 - 25 = 119

let outFunct = outerFunction(4);
let diffSqu = outFunct(11);
console.log(diffSqu); // 16 - 121 = -105 = 105
```